CUBE: A Unified Standard and Benchmark Suite for Evaluating Generalist Agents

Anonymous Author(s)
Affiliation
Address
email

Abstract

The agentic benchmark landscape now exceeds 600 environments and is still growing, yet each one ships its own provisioning logic, action space, tool definitions, and evaluation harness. The resulting $N \times M$ integration tax blocks cross-benchmark research, including the multi-environment RL post-training that recent work identifies as the strongest path to generalist capability. We introduce **CUBE** (Common Unified Benchmark Environments): a Python-first standard for tasks, benchmarks, tools, and a three-tier resource lifecycle, with an optional JSON-RPC interface for non-Python clients. Any runtime that drives the `Task` gym API is a valid harness; AgentBeats and NeMo Gym already consume CUBE benchmarks. The reference harness adds auto-provisioning across Docker, VM, and live backends, a registry of 10 wrapped benchmarks across web, computer-use, SWE, and terminal modalities, and a trajectory judge that attributes per-episode failures to agent, tool, or benchmark causes via $K{=}3$ cross-judging with post-hoc evidence verification. We evaluate 4 frontier models on 8 CUBEs with tool stacks held constant per modality, exposing capability profiles that per-benchmark leaderboards do not surface and pinpointing where failures live: roughly 82 % trace to the LLM itself, with the rest concentrating on specific tool stacks and on wrapper bugs that the judge surfaces and that we then fix in place. Wrapping a new benchmark reduces to a few typed classes plus metadata, and a Claude skill turns even that into an afternoon of work.

1 Introduction

The number of agentic benchmarks has grown from a handful to over 600 in five years (Figure 1). More than 100 new benchmarks were published in Q1 2026 alone, spanning web navigation, computer use, software engineering, embodied control, scientific discovery, and a dozen other categories.[1] This growth reflects genuine scientific demand: agentic evaluation is hard, domain coverage is uneven, and researchers keep finding gaps that require new environments. The proliferation is a sign of health, not pathology. The problem is that each benchmark arrives as an island.

Each benchmark ships its own scaffolding. WebArena requires a set of web servers provisioned from a specific snapshot. SWE-bench requires per-task Docker containers cloned from a repository matrix. OSWorld requires a pre-provisioned virtual machine image with a running desktop. WorkArena requires a live ServiceNow instance. Every benchmark defines its own action space, names its own tools, makes its own assumptions about how an agent is called, and provides its own (often fragile) evaluation harness. A team that wants to evaluate a single agent across five benchmarks pays five independent integration costs. The cost multiplies with models: N models on M benchmarks means $N \times M$ integration points. No wonder cross-benchmark results are rare and comparisons are almost always informal.

This fragmentation blocks more than evaluation. Recent work argues that future agentic systems should be designed for generality across diverse environments [Bandel et al., 2026], and large-scale

[1] Corpus curated via deep search of arxiv listings; each entry requires an interactive environment, links to a published paper, and has been individually verified.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

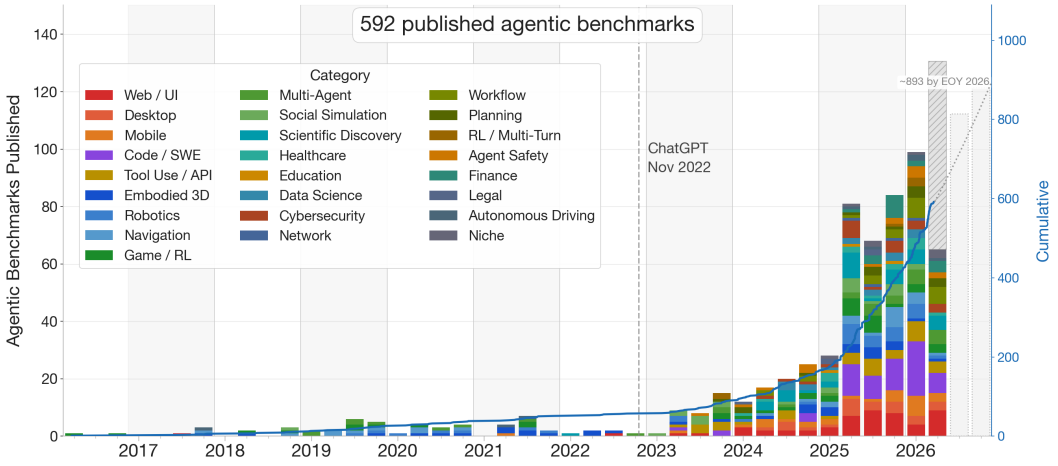

Figure 1: Agentic benchmarks published per quarter, 2016–2026, by category (cumulative count on right axis). Dashed line: ChatGPT (November 2022). Q2 2026 is ≈37% complete; the gray hatched bar estimates remaining Q2 publications from a monthly linear trend since January 2023. Outlined bars and dotted projection line show Q3–Q4 2026 forecasts from the same model. See §5 for corpus details.

reinforcement learning post-training has emerged as a primary route to capability gains in frontier models [Khatri et al., 2025]: every additional environment the model trains on adds a dimension of generalization it could not acquire from any one benchmark alone. With 600+ agentic benchmarks now in existence, the ideal training curriculum already exists on paper. But sampling from hundreds of environments at training time requires every benchmark to expose the same interface, the same resource lifecycle, and the same tool vocabulary. No such interface exists today. Every custom integration is a constraint on the training distribution, and a narrower distribution caps what can be learned. The integration tax on evaluation is the ceiling on generalist capability tomorrow.

We introduce **CUBE** (Common Unified Benchmark Environments), addressing this bottleneck through four contributions.

The CUBE Standard (Section 3): a Python-first set of abstractions for tasks, benchmarks, tools, and resources, with a three-tier resource lifecycle for transparent auto-provisioning across Docker, VM, and cloud backends, and an optional JSON-RPC interface for non-Python clients. The standard also introduces a composite benchmark class that subsets and combines existing CUBEs as a difficulty-calibrated cross-modality benchmark.

A reference harness (Section 4): although the standard is harness-agnostic and interoperates with AgentBeats and NeMo Gym, we provide a reference harness with parallel execution, OpenTelemetry tracing, trajectory logging for RL training data, and a trajectory judge that attributes per-episode failures to agent, tool, or benchmark causes, making cross-benchmark failure analysis tractable at scale.

A wrapped corpus and registry (Section 5): 10 CUBEs spanning web, computer use, software engineering, and terminal modalities, with automated compliance verification and a live registry. At the time of writing the registry includes a cube for MiniWoB [Shi et al., 2017], WebArena [Zhou et al., 2024], WorkArena [Drouin et al., 2024], BrowseComp [Wei et al., 2025], OSWorld [Xie et al., 2024], Windows Agent Arena [Bonatti et al., 2025], SWE-bench [Jimenez et al., 2024], SWE-bench Verified [Chowdhury et al., 2024], Terminal-Bench [Merrill et al., 2026], and DRBench [Abaskohi et al., 2025]; and with the release of our claude skill to wrap any benchmark within CUBE, we expect the registry to grow significantly.

Cross-modality evaluation (Section 6): a systematic evaluation of frontier models across web, CUA, and SWE tasks with tooling held constant per modality, exposing capability profiles that per-benchmark reports tend to miss.

Standardization enables research that is otherwise impractical. When tools are first-class objects in the standard, tool design becomes an independent variable: one browser tool drives WebArena, WorkArena, and MiniWoB, making tool ablations honest across benchmark families. When composites are a core abstraction, difficulty can be recalibrated as agents improve without re-engineering anything. When every benchmark speaks the same interface, curriculum training across hundreds of environments is a sampling-policy decision, not an integration project. Section 7 returns to each with evidence.

2 Related Work

Agentic benchmarks. The agent-evaluation landscape now spans hundreds of environments [Yehudai et al., 2025], including multi-environment benchmarks such as AgentBench [Liu et al., 2024], AppWorld [Trivedi et al., 2024], and τ-bench [Yao et al., 2025]; each individual benchmark CUBE wraps embeds strong infrastructure assumptions. WebArena [Zhou et al., 2024] requires a persistent shared server stack; WorkArena [Drouin et al., 2024] requires a live ServiceNow instance; OSWorld [Xie et al., 2024] and Windows Agent Arena [Bonatti et al., 2025] require VM snapshots with a running desktop; SWE-bench [Jimenez et al., 2024, Chowdhury et al., 2024] creates per-task Docker containers from a repository matrix; Terminal-Bench [Merrill et al., 2026] uses hermetic per-task containers; MiniWoB [Shi et al., 2017] runs inside a local browser. These incompatible resource models are the direct source of the $N \times M$ integration tax described in Section 1. BrowserGym [Le Sellier de Chezelles et al., 2025] demonstrates the value of a shared interface within the web modality; CUBE generalizes this approach across modalities and lifts the benchmark interface to include resource provisioning and tool configuration. HAL [Kapoor et al., 2026] surfaces a complementary problem: without controlled tooling, scores on the same benchmark are not comparable across papers. CUBE addresses this directly by making tools first-class objects in the standard, so the tool configuration is part of the benchmark's declared interface.

Platforms and orchestration. Several systems address agentic evaluation and training from complementary angles. The METR Task Standard [METR, 2024] fixes a portable Docker/VM task-image convention with a small lifecycle of hooks; $\sim$200 task families have been wrapped under it, and it is used across METR, UK AISI, and partner organizations. Inspect AI [UK AI Security Institute, 2024] is AISI's evaluation orchestration framework, with 200+ pre-built evaluations, sandboxing, MCP [Anthropic, 2024] tool support, and bridges to coding agents. AgentGym [Xi et al., 2025] bundles 14 environments under a unified HTTP API, primarily as substrate for its AgentEvol training method rather than as a third-party-extensible standard. AgentBeats [AgentBeats Team and Berkeley RDI, 2025] proposes the Agentified Agent Assessment paradigm, in which benchmarks are realized as judge agents communicating over A2A and MCP, and operates a registry of judge agents alongside hosted competitions. NeMo Gym [NVIDIA, 2025] provides high-throughput RL training infrastructure with a microservice architecture and integrations into NeMo RL and OpenRLHF; OpenEnv [Meta PyTorch Team and Hugging Face, 2025] distributes Gymnasium-style environments through the HuggingFace Hub. PipelineRL [Piché et al., 2026] streams weight updates to inference workers to overlap generation and training on long-trajectory rollouts, and Harbor [Shaw, 2025] grew out of Terminal-Bench as a container-based post-training pipeline.

These systems operate at distinct layers: image conventions (METR), evaluation orchestration (Inspect), curated training suites (AgentGym), and RL execution (the rest). CUBE sits at the wrapping layer with a Python-native typed object model for tasks, benchmarks, tools, and the resource lifecycle, a JSON-RPC contract for non-Python clients, a registry for third-party contributions, and reference connectors that expose any CUBE benchmark as a NeMo Gym environment or an AgentBeats green agent.

RL post-training over agentic environments. Reinforcement learning post-training is now central to frontier LLM development and follows predictable scaling laws [Khatri et al., 2025]; recent position work argues that the natural endpoint is generalist agentic systems trained and evaluated across diverse environments [Bandel et al., 2026]. Prior work on multi-environment RL for agents has been limited to bespoke per-project integrations, with custom reset and reward logic per environment and no shared tool vocabulary. A standard task interface, resource lifecycle, and portable tool set are the precondition layer that makes large-scale curriculum training across heterogeneous benchmarks tractable rather than a per-project engineering effort.

3 The CUBE Standard

The CUBE standard serves two audiences. A *downstream user* (a harness, a training platform, or a researcher) needs to run any benchmark and provision its resources through a single uniform interface, without writing benchmark-specific code. A *benchmark author* needs a clear contract so that a single CUBE compliant implementation reaches all downstream users at once. The standard is harness-agnostic and ships as a pure Python library.[2] A central design principle is the *config/make split*: every runtime object (benchmark, task, tool) has a serializable config counterpart that crosses process boundaries safely; calling `.make()` on the config returns the live object that holds a dynamic state and is never serialized.

3.1 Running a CUBE

Running a CUBE is config-first. A downstream user assembles an experiment by composing serializable Python objects: a tool config, a benchmark config, an infra config, and an agent config. The framework separates shared setup from per-task execution: calling `.make(infra)` on the benchmark config provisions shared resources (servers, containers, VMs) once; the resulting task configs are lightweight serializable records that travel to Ray workers, which access or instantiate live environments locally based on their needs. Only serializable handles, (endpoints, IDs, infra config) cross to worker processes via a `runtime_context` dict, never live environments. Workers access live clients locally from those handles. The episode loop (right box) is gym-style: observe, act, repeat until terminal. The key addition over vanilla Gymnasium [Towers et al., 2024] is that `evaluate()` is decoupled from `step()` and can be called at any point independently of action frequency, which matters when agent and evaluator run on different schedules.

Harness side: configure, iterate, dispatch

```python
# serializable configs defines the experiment
tool_cfg = BrowsergymConfig(use_screenshot=True)
wav_cfg = WebArenaVerifiedConfig(tool_config=tool_cfg)
agent_cfg = MyAgentConfig(llm_config=...)
infra_cfg = AzureInfraConfig()  # or AWS, GCP, local

@ray.remote
def run(task_cfg, runtime_ctx, agent_cfg):
  # right box -->
  return run_episode(task_cfg, runtime_ctx, agent_cfg)

# .make() provisions shared resources
# leaving the 'with' block tears them down
with wav_cfg.make(infra_cfg) as bench:
  rc = bench.runtime_context  # endpoints, IDs, infra
  task_cfgs = bench.config.get_task_configs()
  ray.get([run.remote(tc, rc, agent_cfg) for tc in task_
      cfgs])
```

Gym-like Episode loop: one task, one agent

```python
# evaluation running on a remote process
def run_episode(task_cfg, runtime_ctx, agent_cfg):

    task  = task_cfg.make(runtime_ctx)
    agent = agent_cfg.make(task.action_set)
    obs, _ = task.reset()
    done = False
    while not done:
        act = agent.act(obs)
        out = task.step(act)
        obs, done = out.obs, out.done
    return task.evaluate()
```

Resource provisioning. A benchmark declares its infrastructure requirements as a list of `ResourceConfig`: the base image its containers or VMs need, any storage volumes, and setup scripts. When an experiment starts, the harness resolves these against the user's `InfraConfig`, downloads the original benchmark artifacts, and builds or registers a benchmark-ready image on the target provider (local Docker, Azure, AWS, or others). Benchmark authors write their `ResourceConfig` once; users point at any supported infra target and benchmark code does not change. Three structurally different provisioning patterns appear across the corpus: (i) WebArena needs a shared server started once per run: the author starts it in `Benchmark._setup()` and passes the URL to tasks via `RuntimeContext`; (ii) SWE-bench needs a fresh container per task: the harness launches one via `runtime_context['infra']` when the task is constructed. (iii) OSWorld needs a pre-built VM image: the author declares a `VMResourceConfig`; the harness builds it on first use and caches it.

Composite benchmarks and RPC interface. `CompositeBenchmarkConfig` composes any number of existing benchmarks into a single standards-compliant one. Task IDs are prefixed by sub-benchmark name; routing is transparent to callers. Provisioning, RPC server, compliance verification, and registry metadata are all inherited. Composites nest freely. The standard also auto-generates a JSON-RPC 2.0 server with MCP-compatible endpoints from any `Benchmark` subclass, so any MCP client drives a CUBE task without an adapter layer (see Appendix A).

[2] Anonymous repository: `https://anonymous.4open.science/r/cube-standard-123`

3.2 Wrapping a Benchmark

Every CUBE must provide a default tool that defines the agent's action space: the tool is the benchmark's public interface for any downstream agent or training pipeline, so tool selection is the first authoring decision. The standard ships reusable reference tools for each major modality. A BrowserGym-compatible browser tool drives web benchmarks (WebArena, WorkArena, MiniWoB); a desktop tool built on accessibility trees and PyAutoGUI drives CUA benchmarks (OSWorld, Windows Agent Arena); terminal tools drive SWE and scripted benchmarks. For benchmarks where tool design is itself a research variable, the separation between tool and task means a benchmark ships a sensible default while staying configurable: swapping the tool for the same benchmark produces directly comparable scores because the environment is held constant. Once a tool is chosen, authoring reduces to three steps: (1) declare benchmark metadata, task metadata and resources; (2) implement `reset()` and `evaluate()` on the Task subclass, and (3) ship a debug suite.

Tools. Custom tools follow the `@tool_action` pattern: the decorator derives the full action schema from the method signature and docstring, so no separate specification is needed. `ToolConfig.make(container)` instantiates the tool after its container is live; switching to a different backend means swapping the `InfraConfig` passed to `.make(infra)`; tool, task and benchmark code are unchanged.

Metadata and task execution info. A benchmark declares `BenchmarkMetadata` (name, version, description, task count, reset isolation mode) and per-task `TaskMetadata` (id, split, container config). Heavy per-task data (patches, archives, long problem statements, binaries) lives on a `TaskExecutionInfo` subclass populated lazily on workers, keeping initialization flat at scale. `BenchmarkConfig.install()` writes that data to a per-task cache (`$HOME/.cube/...`) once per worker; later runs read it from disk.

Evaluation. `task.evaluate()` is the benchmark author's primary scientific contribution. It returns a reward in $[0, 1]$ with a structured info dict. The base class calls it at episode end, but the harness can also call it independently to support partial-credit scoring or judge-agent analysis. Privileged evaluation data (ground-truth patches, evaluator scripts, internal state) lives in the `task.execution_info` field, kept off the agent's observation and tool surface; an opt-in `task.get_privileged_info()` hook exposes it to judge agents or debug clients when needed.

Debug suite and robustness. Every CUBE ships two LLM-free contracts that the standard requires and the compliance check verifies. `get_debug_benchmark()` returns a `BenchmarkConfig` pre-filtered to a small set of tasks with deterministic, known-correct solutions. `make_debug_agent(task_id)` returns a scripted agent guaranteed to reach reward 1.0 on those tasks without calling any language model. Running the debug suite against real infrastructure proves that the provisioning pipeline, tool execution, and evaluation logic all work end-to-end, with no model-in-the-loop ambiguity. The registry runs this suite post-merge whenever an entry changes; the result feeds the entry's status badge on the registry site. Locally, `cube test <name>` runs the same suite, so authors catch broken environments before pushing.

Authoring tools. The standard ships with layer-by-layer specifications in `openspec/specs/`; each spec includes Invariants and Gotchas sections that enumerate the most common authoring mistakes. Two Claude Code skills exploit these specifications directly. `/new-cube` generates a complete CUBE package from a benchmark description: task subclass, tool subclass, benchmark subclass, debug suite, serializable configs, and `pyproject.toml`, all consistent with the standard's invariants. `/review-cube` audits an existing implementation against those invariants and reports violations before the registry's compliance check runs. These skills are the primary onboarding path for new contributors and reduce the typical wrapping effort for an existing benchmark from days to hours.

4 A Reference Harness

The CUBE standard is harness-agnostic: any runtime that can instantiate a `BenchmarkConfig` and drive the `Task` gym API is a valid harness. AgentBeats and NeMo Gym both consume CUBE benchmarks directly, and a reference NeMo Gym connector in `cube-standard` exposes any benchmark as a Gymnasium environment for RL frameworks in roughly 100 lines[3]. We provide a reference harness that runs the full stack end-to-end and serves as the execution engine for the experiments

[3] https://anonymous.4open.science/r/cube-standard-123/src/cube/integrations/nemogym.py

in Section 6. Adopting CUBE does not require adopting our harness. The harness contributes three pieces: a generalist agent, a Ray-based experiment runner with trajectory logging, and a trajectory judge that automates per-episode failure analysis.

Generalist agent. `Genny`, the agent used for the experiments in Section 6[4], consumes any CUBE-compliant action set without per-benchmark code and offers two cache-friendly context modes. In *growing-history* mode, each step appends one (observation, action) pair to a stable cacheable prefix; we use this for SWE agents, where tool-call results themselves carry the state the agent needs to retain. In *rolling-summary* mode, a separate summarizer LLM produces a per-step summary stored as its own message; the prompt keeps the accumulated summaries plus the current observation while older raw observations are dropped, so the prefix cache extends cleanly across steps. This mode covers UI-based agents (web, CUA), where retaining every screenshot or accessibility tree would exhaust the context window within a few steps.

Experiment runner. The runner schedules episodes with Ray. Each worker receives a serialized `Episode` (containing `TaskConfig`, `AgentConfig`, and the benchmark's `runtime_context`), calls `make()` locally to instantiate the live task, tool, and agent, runs the episode, and ships a `Trajectory` back to the coordinator. No live object crosses a process boundary. The `Benchmark` itself stays on the driver. The coordinator enforces per-step timeouts and retries transient failures (`FAILED`, `CANCELLED`, `STALE`), tracking each episode through `QUEUED` $\rightarrow$ `RUNNING` $\rightarrow$ `COMPLETED`|`FAILED`. Parallelism is bounded by infra capacity and the Ray worker pool.

Trajectories are persisted to a per-episode file store that feeds RL post-training pipelines, and LiteLLM token counts give the cost-per-task figures used in Section 6.

4.1 Trajectory judge

Binary pass/fail tells you how often an agent fails, not why. With experiments running hundreds of episodes across multiple modalities, manual failure triage does not scale. As a remedy, the harness ships a per-episode judge that runs an external coding agent (Claude Code via `claude-agent-sdk`) over the recorded trajectory and emits a structured `JudgeOutput`: narrative analysis, outcome, primary blame category with confidence, additional blames, supporting evidence, and a failure hypothesis. Beyond diagnosing agent limitations, the blame distribution serves a second role: it validates wrapper faithfulness. A low benchmark-side blame rate (`env_failure`, `eval_brittle`, `task_unclear`) is evidence that the CUBE wrappers are not introducing systematic evaluation artifacts.

LLM-as-judge [Zheng et al., 2023, Gu et al., 2024] remains unreliable in general: judges underperform on objectively-correct response comparisons [Tan et al., 2025], hallucinate evidence and miscalibrate confidence [Jung et al., 2025], and naive judges systematically misclassify agent-trajectory outcomes [Lù et al., 2025]. Closer to our setting, MAST [Cemri et al., 2025] derived a 14-mode failure taxonomy for multi-agent traces with κ=0.88 agreement; the CUBE judge takes the same evidence-grounded approach at the single-agent level, with a 3-bucket attribution into agent, tool, or benchmark causes designed for use as a corpus-wide faithfulness signal across a 10-benchmark suite.

Grounded inputs. Per experiment, an Opus-class agent runs once (turn budget 80) to assemble a *context pack*: pointers and summaries for the agent prompt, tool definitions, task description, reward function, and infra entrypoints. A Sonnet-class judge then runs per episode (turn budget 40) with the context pack and the trajectory directory, invoked through `claude-agent-sdk`. Crucially, the judge reads files via its own tool loop rather than receiving a stuffed prompt, so attributions are grounded in real source: it can open a specific step's accessibility tree or read the actual reward function.

Constrained outputs and reliability checks. The judge selects the *primary blame* from a fixed 10-category taxonomy (Appendix C) and reports `none` rather than inventing a cause; any non-`none` blame must be supported by step-indexed verbatim quotes pulled from the trajectory, and confidence is reported on a 0–5 scale. Two reliability layers sit on top. First, *post-hoc evidence verification*: every quote is checked against the persisted step file, mismatches (literal or fuzzy) trigger a confidence downgrade and catch the most common hallucination mode in our setting (synthesising a plausible-looking quote). Second, *cross-judging with* $K = 3$: independent judges vote, the modal blame is reported, and quantitative aggregation is restricted to judgements with at least two-of-three agreement

[4] https://anonymous.4open.science/r/cube-harness-123/src/cube_harness/agents/genny.py

after verification. In practice $\geq 97\,\%$ of episodes reach majority agreement on the primary blame after this filter (Appendix E); transcripts are persisted with each episode record, leaving the remaining disputed cases auditable.

5 Benchmark Corpus and Registry

Our initial corpus spans web automation (WorkArena, WebArena-Verified, MiniWoB, BrowseComp), computer use (OSWorld, Windows Agent Arena), software engineering and terminal (SWE-bench Verified, SWE-bench Live, Terminal-Bench), and enterprise research (DRBench), totalling 5,642 tasks across four structurally distinct provisioning models (local browser, shared multi-service Docker stack, per-task Docker container, virtual machine, and live external service). The set was assembled to stress the standard from every direction: each modality forces a different combination of action space, infrastructure, and evaluation logic, and covering them in one corpus is what surfaced the abstractions in Sections 3 and 4. For the experiments in Section 6 we draw from the eight CUBEs in the Web, CUA, and SWE blocks; BrowseComp and DRBench are omitted to keep compute tractable. Per-CUBE task counts and backend details are tabulated in Appendix B.

A registry for a decentralized corpus. Any author can publish a CUBE in their own repository and ship it through any Python index. The registry is a lightweight file-based metadata index, GitHub-orchestrated rather than a hosted service[5]. Each entry is a small YAML record (name, version, authors, license, package link) submitted via pull request; the registry stores no benchmark code, only the pointer to a repository implementing that specific CUBE.

Automated compliance on submission. Submitting a CUBE triggers GitHub Actions compliance checks. A *quick check* (schema validation, package import, action introspection) runs on every PR in roughly two minutes. A *full check* post-merge executes the debug agent on the debug task set; a CUBE is compliant only if the agent reaches reward 1.0 on every debug task. Passing both earns a compliance badge; failing the full check flags the entry degraded until a fix is submitted. This lets the registry accept third-party CUBEs at scale without trusting any individual submitter.

6 Experiments

Our experiments answer three questions. First, can a single generalist agent configuration produce useful baselines across the corpus (§6.2)? Second, where do its failures originate: the LLM, the agent scaffolding, the tool stack, or the wrapped benchmark itself (§6.3)? Third, and most critically for an evaluation suite: are the CUBE wrappers faithful implementations of the underlying benchmarks? Reproducing published leaderboard numbers proved too noisy a signal for this purpose (Appendix D); we used the judge's benchmark-side blame rate instead, which enabled iterative wrapper debugging and yielded direct evidence of faithfulness once benchmark-side blame converged to a low share.

6.1 Setup

Models. We evaluate four frontier models spanning two providers, with a small/large split per provider: Anthropic Claude Haiku and Claude Sonnet, OpenAI GPT-5.4-mini and GPT-5.4. All calls go through the harness's LiteLLM gateway with default sampling parameters.

Agent. We use Genny (§4) as the single generalist agent across the corpus. Genny operates in two modes selectable per benchmark: rolling summaries for UI-based tasks (web, CUA), where retaining every screenshot or accessibility tree would saturate the context window; growing history for SWE and terminal tasks, where tool-call results carry the state the agent needs to retain. We do not tune a per-benchmark prompt; the only benchmark-aware choices are the context-management mode and the registered tool stack.

Tool stacks. One tool stack per modality, held constant across all benchmarks in that modality. Web: BrowserGym's high-level action set. CUA: accessibility-tree input with PyAutoGUI for action output. SWE: a bash tool with structured stdout/stderr capture. Holding tooling constant per modality is what lets per-modality results be compared directly across benchmarks.

[5] registry URL kept private during reviewing

Table 1: Pass rate (%) per model and CUBE on the evaluation subsets. Best per row is shown in bold.

		Anthropic		OpenAI	
CUBE (subset)	**# Tasks**	**Haiku 4.5**	**Sonnet 4.6**	**GPT-5.4-mini**	**GPT-5.4**
WorkArena L1	33×10	31.7 ±2.6	**65.6 ±2.6**	27.3 ±2.5	53.6 ±2.7
WorkArena L2	235	15.6 ±2.4	**35.5 ±3.1**	5.2 ±1.4	35.7 ±3.1
WebArena-Verified	381	—	**14.8 ±1.8**	—	9.0 ±1.5
MiniWoB	125×5	**67.2 ±1.9**	59.4 ±2.0	52.5 ±2.0	52.5 ±2.0
OSWorld (PyAutoGUI)	360	43.2 ±2.6	**49.4 ±2.6**	17.3 ±2.0	28.6 ±2.4
OSWorld (Computer13)	360	42.1 ±2.6	**51.6 ±2.6**	27.5 ±2.4	30.7 ±2.4
Win. Agent Arena (PyAutoGUI)	154	41.5 ±4.0	**61.2 ±3.9**	25.7 ±3.5	37.5 ±3.9
Win. Agent Arena (Computer13)	154	**32.9 ±3.8**	32.2 ±3.8	23.7 ±3.4	27.0 ±3.6
SWE-bench Verified	500	43.0 ±2.2	**65.1 ±2.1**	45.4 ±2.2	58.2 ±2.2
SWE-bench Live (lite-gold)	223	20.2 ±2.7	**38.6 ±3.3**	22.3 ±2.8	35.4 ±3.2
TerminalBench	89	14.0 ±3.7	**29.5 ±4.8**	12.4 ±3.5	11.4 ±3.4

Benchmark subsets. We use full sets where tractable. WorkArena is restricted to L1 and L2 (L3 omitted); SWE-bench Live [Zhang et al., 2025] to *lite-gold-226*, the lite-split tasks whose gold patches pass evaluation verbatim; BrowseComp and DRBench are excluded. Subset sizes appear in Table 1.

Reporting. For each (model, CUBE) pair we report pass rate as the primary metric, average number of steps, and average dollar cost per task as secondary metrics. Cost is computed from token counts recorded by the LiteLLM callback multiplied by published model rates. Each task is hard-capped at 150 steps and at a per-task budget of $1 (small models: Haiku 4.5, GPT-5.4-mini) or $3 (large models: Sonnet 4.6, GPT-5.4). Tasks that exhaust either limit are counted as failures.

Faithfulness validation. We use the judge's benchmark-side blame rate as the primary faithfulness signal: when `env_failure`, `eval_brittle`, and `task_unclear` are rare, the wrapper is not the bottleneck. This signal drove several rounds of wrapper debugging before the final runs, each round flagging a specific evaluator error, provisioning issue, or tool misconfiguration that we then fixed (Appendix D).

6.2 Baseline pass rates

Table 1 reports per-model, per-CUBE pass rate across the subsets defined above.

Sonnet 4.6 leads on most CUBEs, doubling the small models' rates on the harder subsets (WorkArena L2, SWE-bench Live, TerminalBench); on easier or saturating subsets (WorkArena L1, MiniWoB) the gap closes and Haiku occasionally tops the column. SWE shows the steepest capability cliff: small models compress the SWE ranking sharply, suggesting long-horizon coding benefits disproportionately from capability rather than scaffolding. These are *platform measurements*, not SOTA claims: Genny is a single generalist with one prompt and one tool stack per modality, no per-benchmark tuning, and a flat $1/$3 budget cap. Bespoke pipelines that tune prompts and tools per benchmark routinely score higher; our goal is comparable numbers under a fixed configuration, not leaderboard positions.

6.3 Where do failures originate?

Aggregating the structured judge outputs across the corpus answers both the accountability question and the faithfulness question. The dominant cause of failure is the agent itself: limitations in the underlying LLM's reasoning, planning, or context management account for roughly **82 %** of attributions, with the remaining **18 %** split between benchmark-side (**11 %**) and tool-side (**6 %**) issues. Figure 2 breaks the distribution down by benchmark using the judge's nine failure-attribution categories, grouped visually into agent-side (blue shades), tool-side (green), and benchmark-side (rose) buckets.

Two patterns stand out. On CUA and SWE the agent-side share exceeds 80 %, with `model_capability` alone accounting for roughly four-fifths of attributions: closing this gap

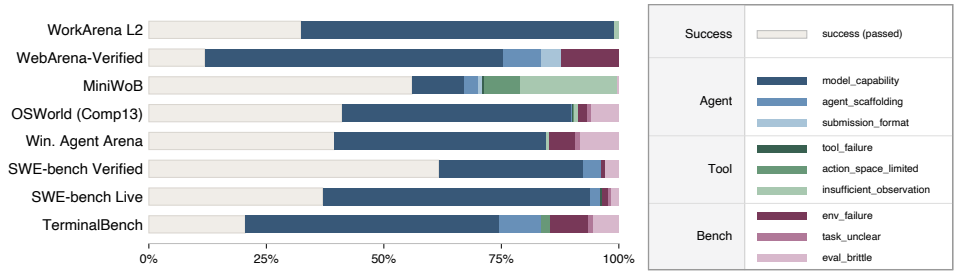

Figure 2: Blame distribution per benchmark. Each bar spans 100 % of tasks: the pale left segment is the pass rate (averaged over Sonnet 4.6 and GPT-5.4); the remainder is divided among the 9 failure-attribution categories (excluding `none`), scaled proportionally to the judge's blame counts.

is the LLM-research problem, not an evaluation one. MiniWoB is the single benchmark where tool-side blame dominates: nearly 65 % of its failures hit `action_space_limited` or `insufficient_observation` because the BrowserGym `bid` action set cannot express the pixel-level mouse interactions that roughly 30 % of MiniWoB tasks require, illustrating that tool configuration is an independent variable. The benchmark-side share concentrates on Windows Agent Arena and TerminalBench; each flagged case is a queued issue we are iterating on (Appendix F).

6.4 Qualitative findings from the judge

Beyond the structured fields, the judge's free-form *analysis* acts as a curated review of the corpus. Three recurring failure shapes show up repeatedly across our 8 CUBEs: capability gaps, where the relevant evidence is in the trajectory but the agent fails to connect it; scaffolding pathologies, typically deterministic loops or context exhaustion that the agent cannot escape on its own; and wrapper or evaluator bugs that only become visible once several episodes are aggregated. Appendix F works through one three-judge case per category. The benchmark-side case (VLC `vlcrc` cluster, Table 7) is representative: five disjoint episodes flagged the same Python escape-sequence trap, and a one-line `os.path.join` fix flipped all five from 0.0 to 1.0 on resume.

7 Discussion

Wrapping a benchmark is now a small, well-scoped job. Under the standard, integrating a new benchmark reduces to implementing a handful of typed classes (`Task`, `Benchmark`, `ToolConfig`) plus declarative metadata, and inheriting the shared resource lifecycle, JSON-RPC interface, and compliance test suite for free. In our own experience the per-benchmark wrapping cost collapsed from weeks of bespoke harness integration to roughly a day of focused work. We further release a `/new-cube` Claude skill that scaffolds the wrapper, generates the metadata, and runs the compliance checks interactively; with this tool, a benchmark author who knows their own benchmark can produce a passing CUBE in an afternoon. We expect this combination of a small, opinionated standard and an LLM-driven wrapping assistant to be the dominant lever for growing the registry.

The judge turns every run into a debugging signal. A recurring pattern emerged across our 8 CUBEs: several independent failed episodes converge on the same root cause, and the fix is usually a single line of wrapper or evaluator code (the VLC cluster in §6.4 is one such case). Beyond attributing failures to the agent, the judge therefore functions as a continuous review of the wrappers, and of the underlying benchmarks themselves. Several `should_have_been_rewarded` verdicts in our runs pointed at genuine ambiguity in task descriptions or evaluator logic, artefacts that the original benchmark authors are in a position to refine. A shared judge protocol thus turns evaluation runs into a shared queue of triaged, evidence-backed issues for the entire benchmark ecosystem.

Limitations. Streaming actions/observations and multi-agent settings are not yet supported and are the next protocol extensions we plan to ship; dataset-only benchmarks remain out of scope. "Tools held constant" is harness-enforced rather than standard-enforced; the compliance suite is a floor, not a correctness guarantee; faithfulness is bounded by judge accuracy (Appendix E); claims extend only to the 8 CUBEs evaluated.

References

Amirhossein Abaskohi, Tianyi Chen, Miguel Muñoz-Mármol, Curtis Fox, Amrutha Varshini Ramesh, Étienne Marcotte, Xing Han Lù, Nicolas Chapados, Spandana Gella, Peter West, Giuseppe Carenini, Christopher Pal, Alexandre Drouin, and Issam H. Laradji. Drbench: A realistic benchmark for enterprise deep research. *arXiv preprint arXiv:2510.00172*, 2025. URL https://arxiv.org/abs/2510.00172.

AgentBeats Team and Berkeley RDI. Agentbeats: Towards agentified agent assessment (aaa). https://agentbeats.dev, 2025. Platform for standardized agent evaluation using the Agentified Agent Assessment paradigm.

Anthropic. Model context protocol (mcp), 2024. URL https://modelcontextprotocol.io/. Accessed: 2026-01-20.

Elron Bandel, Asaf Yehudai, Alexandre Lacoste, Avijit Ghosh, Graham Neubig, Margaret Mitchell, Michal Shmueli-Scheuer, and Leshem Choshen. Position: Agentic systems should be general. In *ICLR 2026 Workshop on Agents in the Wild*, 2026. URL https://openreview.net/forum?id=CbJpizP0vJ.

Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Bucker, Lawrence Keunho Jang, and Zheng Hui. Windows agent arena: Evaluating multi-modal OS agents at scale. In *Forty-second International Conference on Machine Learning (ICML)*, 2025. URL https://openreview.net/forum?id=W9s817KqYf.

Mert Cemri, Melissa Z. Pan, Shuyi Yang, et al. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2025. URL https://arxiv.org/abs/2503.13657.

Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Evan Mays, Rachel Dias, Marwan Aljubeh, Mia Glaese, Carlos E. Jimenez, John Yang, Leyton Ho, Tejal Patwardhan, Kevin Liu, and Aleksander Madry. Introducing SWE-bench verified. OpenAI announcement, 2024. URL https://openai.com/index/introducing-swe-bench-verified/.

Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. Workarena: How capable are web agents at solving common knowledge work tasks? In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 11642–11662, 2024. URL https://proceedings.mlr.press/v235/drouin24a.html.

Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Yuanzhuo Wang, Wen Gao, Lionel Ni, and Jian Guo. A survey on LLM-as-a-judge. *arXiv preprint arXiv:2411.15594*, 2024. URL https://arxiv.org/abs/2411.15594.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

Jaehun Jung, Faeze Brahman, and Yejin Choi. Trust or escalate: LLM judges with provable guarantees for human agreement. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL https://openreview.net/forum?id=UHPnqSTBPO.

Sayash Kapoor, Benedikt Stroebl, Peter Kirgis, Nitya Nadgir, Zachary S. Siegel, Boyi Wei, Tianci Xue, Ziru Chen, Felix Chen, Saiteja Utpala, Franck Ndzomga, Dheeraj Oruganty, Sophie Luskin, Kangheng Liu, Botao Yu, Amit Arora, Dongyoon Hahm, Harsh Trivedi, Huan Sun, Juyong Lee, Tengjun Jin, Yifan Mai, Yifei Zhou, Yuxuan Zhu, Rishi Bommasani, Daniel Kang, Dawn Song, Peter Henderson, Yu Su, Percy Liang, and Arvind Narayanan. Holistic agent leaderboard: The missing infrastructure for ai agent evaluation. In *The Fourteenth International Conference on Learning Representations (ICLR)*, 2026. URL https://openreview.net/forum?id=vUaY1t64ZZ.

Devvrit Khatri, Lovish Madaan, Rishabh Tiwari, Rachit Bansal, Sai Surya Duvvuri, Manzil Zaheer, Inderjit S. Dhillon, David Brandfonbrener, and Rishabh Agarwal. The art of scaling reinforcement learning compute for LLMs. *arXiv preprint arXiv:2510.13786*, 2025. URL https://arxiv.org/abs/2510.13786.

Thibault Le Sellier de Chezelles, Maxime Gasse, Alexandre Lacoste, Massimo Caccia, Alexandre Drouin, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omidi Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Graham Neubig, Quentin Cappart, Russ Salakhutdinov, and Nicolas Chapados. The browsergym ecosystem for web agent research. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL https://openreview.net/forum?id=5298fKGmv3. Expert Certification.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating LLMs as agents. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL https://openreview.net/forum?id=zAdUB0aCTQ.

Xing Han Lù, Amirhossein Kazemnejad, Nicholas Meade, Arkil Patel, Dongchan Shin, Alejandra Zambrano, Karolina Stańczak, Peter Shaw, Christopher Pal, and Siva Reddy. Agentrewardbench: Evaluating automatic evaluations of web agent trajectories. In *Conference on Language Modeling (COLM)*, 2025. URL https://openreview.net/forum?id=fQcUZMPIvu.

Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. In *The Fourteenth International Conference on Learning Representations (ICLR)*, 2026. URL https://openreview.net/forum?id=a7Qa4CcHak.

Meta PyTorch Team and Hugging Face. Openenv: An interface library for rl post training with environments, 2025. URL https://github.com/meta-pytorch/OpenEnv. GitHub repository.

METR. METR task standard, 2024. URL https://github.com/METR/task-standard. Portable Docker/VM-based standard for agent task environments; ~200 task families across METR, UK AISI, and partners.

NVIDIA. NeMo Gym: An open source library for scaling reinforcement learning environments for LLMs. https://github.com/NVIDIA-NeMo/Gym, 2025. GitHub repository.

Alexandre Piché, Ehsan Kamalloo, Rafael Pardinas, Xiaoyin Chen, and Dzmitry Bahdanau. PipelineRL: Faster on-policy reinforcement learning for long sequence generation. *Transactions on Machine Learning Research*, 2026. ISSN 2835-8856. URL https://arxiv.org/abs/2509.19128.

Alex Shaw. Harbor Framework, November 2025. URL https://github.com/laude-institute/harbor.

Tianlin Shi, Andrej Karpathy, Linxi Fan, Jonathan Hernandez, and Percy Liang. World of bits: An open-domain platform for web-based agents. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 3135–3144, 2017. URL https://proceedings.mlr.press/v70/shi17a.html.

Sijun Tan, Siyuan Zhuang, Kyle Montgomery, William Y. Tang, Alejandro Cuadron, Chenguang Wang, Raluca Ada Popa, and Ion Stoica. JudgeBench: A benchmark for evaluating LLM-based judges. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL https://openreview.net/forum?id=G0dksFayVq.

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024. URL https://arxiv.org/abs/2407.17032.

Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (ACL)*, pages 16022–16076, 2024. URL https://aclanthology.org/2024.acl-long.850/. Best Resource Paper Award.

UK AI Security Institute. Inspect AI: Framework for large language model evaluations, 2024. URL https://github.com/UKGovernmentBEIS/inspect_ai. Open-source evaluation framework with 200+ pre-built evaluations.

Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. Browsecomp: A simple yet challenging benchmark for browsing agents, 2025. URL https://arxiv.org/abs/2504.12516.

Zhiheng Xi, Yiwen Ding, Wenxiang Chen, Boyang Hong, Honglin Guo, Junzhe Wang, Xin Guo, Dingwen Yang, Chenyang Liao, Wei He, Songyang Gao, Lu Chen, Rui Zheng, Yicheng Zou, Tao Gui, Qi Zhang, Xipeng Qiu, Xuanjing Huang, Zuxuan Wu, and Yu-Gang Jiang. AgentGym: Evaluating and training large language model-based agents across diverse environments. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (ACL)*, pages 27914–27961, 2025. URL https://aclanthology.org/2025.acl-long.1355/.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024. URL https://openreview.net/forum?id=tN61DTr4Ed.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik R. Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL https://openreview.net/forum?id=roNSXZpUDN.

Asaf Yehudai, Lilach Eden, Alan Li, Guy Uziel, Yilun Zhao, Roy Bar-Haim, Arman Cohan, and Michal Shmueli-Scheuer. Survey on evaluation of LLM-based agents, 2025. URL https://arxiv.org/abs/2503.16416.

Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, Junhao Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, Elsie Nallipogu, Qingwei Lin, Yingnong Dang, Saravan Rajmohan, and Dongmei Zhang. SWE-bench goes live!, 2025. URL https://arxiv.org/abs/2505.23419.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023. URL https://openreview.net/forum?id=uccHPGDlao.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL `https://openreview.net/forum?id=oKn9c6ytLx`.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: §1 enumerates four concrete contributions (standard, harness, corpus and registry, cross-benchmark experiments) and states the scope explicitly (10 CUBEs, three modalities). §3–7 each address one contribution. No aspirational claim is left ungrounded.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: §7 (Discussion) closes with a dedicated Limitations paragraph covering: streaming actions/observations and multi-agent settings not yet supported (flagged as the next protocol extensions on the roadmap); dataset-only benchmarks out of scope; "tools held constant" being harness-enforced rather than standard-enforced, so cross-benchmark comparability requires researchers to use the same `ToolConfig`; the compliance suite establishing a floor (debug tasks pass) rather than a correctness guarantee; the judge-based faithfulness signal being bounded by judge accuracy, with the meta-analysis in Appendix E reporting $\geq 97\%$ majority agreement but lower unanimous agreement on borderline cases; and capability claims extending only to the 8 CUBEs evaluated.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.

- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper presents no theoretical results. CUBE is a software standard and empirical benchmark study; all claims are engineering or empirical.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: §6.1 specifies all evaluation details: models (4 frontier models, 2 providers), tool configuration (one per modality, held constant), agent (Genny §4), task subsets (predefined splits documented per CUBE), per-step timeout, and budget cap. The harness code is available at the anonymous repository. Task subset IDs are fixed and documented.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.

- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: The CUBE standard and reference harness are publicly available at the anonymous repository linked in §3 (anonymous.4open.science, browsable without login). No new dataset is introduced; all benchmarks used are existing published resources installed at evaluation time. Installation instructions and evaluation scripts are included in the repository.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: §6.1 documents the model family and provider for each of the four evaluated systems, the tool configuration per modality, the agent (Genny, §4) and its two context-management modes, the task subsets used (full sets or pre-defined splits with rationale), the per-task step cap (150 steps), and the per-task budget cap ($1 for small models, $3 for large). No hyperparameters are tuned; default sampling parameters are used throughout.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: Table 1 reports pass rate $\pm$ 1 binomial standard error ($\sqrt{p(1-p)/n}$) for each (model, CUBE) pair, where n is the effective task count (tasks $\times$ seeds where multiple seeds are used). The metric is task-binary (pass/fail), so the binomial SE is the appropriate measure of task-level variance within each evaluation subset.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: §6.1 (*Reporting*) states the per-task budget cap ($1 for Haiku/GPT-5.4-mini, $3 for Sonnet/GPT-5.4) and the 150-step cap; cost is computed from LiteLLM token counts multiplied by published model rates. All evaluation calls standard cloud APIs; no GPU hardware is required. Total experimental spend across the four model providers was on the order of $20,000 for the full set of (model, CUBE) combinations reported in Table 1.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work introduces an evaluation infrastructure standard. No harmful data is collected or released, no human subjects are involved, and no privacy-sensitive information is processed. All wrapped benchmarks are used in accordance with their respective licenses.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [N/A]

    Justification: CUBE is an interoperability standard and reference harness for evaluation infrastructure; it releases no model weights, agents, training data, or generative outputs. The broader-impact surface lies with the agents being evaluated through CUBE rather than with the standard itself, and is appropriately the concern of those agent-system papers. The work makes no foundation-model or deployment claim that would warrant a societal-impact section in the paper.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: No model weights, generative models, or scraped datasets are released. CUBE ships only a Python interoperability standard, a reference harness, and a registry of pointers to existing benchmark packages.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Each wrapped benchmark is cited with its original paper in §2 (*Agentic benchmarks*) and listed with backend details in Appendix B. Licenses for all upstream benchmarks are recorded in the registry YAML entry for each CUBE. The CUBE standard and reference harness are released under the Apache 2.0 license, stated in the repository.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: Three new assets are introduced and documented: (1) the CUBE standard Python package (§3–4), with full API documentation in the anonymous repository; (2) the reference harness (§4), documented in the same repository; (3) the CUBE registry (§5), a GitHub-hosted file-based index with submission instructions and a compliance CI pipeline. All are available at submission time via the anonymous link in §3.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.

- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: No crowdsourcing or research with human subjects is involved. All evaluation uses automated, environment-based judges; no human annotators or participants are used.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: No human subjects are involved; IRB approval is not applicable.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [Yes]

    Justification: LLMs are a core methodological component in three roles: (1) the evaluated systems, where the paper measures frontier LLM-based agents across the corpus; (2) the trajectory judge (§4.1), where a Claude Code agent reads recorded episodes and produces structured failure attributions; (3) the `/new-cube` and `/review-cube` authoring skills (§3), where Claude Code agents generate and audit CUBE boilerplate. All three roles are described in the paper.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.

Appendix

A JSON-RPC interface details

The standard auto-generates a JSON-RPC 2.0 server from any `Benchmark` subclass. The server exposes two endpoints whose names and payload shape match MCP's `tools/list` and `tools/call`, so any MCP-compatible client (LLM tool-call loop, IDE plugin, evaluation harness) can drive a CUBE task without an adapter layer.

Endpoint: `tools/list`. Returns the action schema derived from the task's default tool: one entry per `@tool_action`-decorated method, with name, description, and JSON Schema for the arguments. The schema is generated at task construction time and is static for the lifetime of the episode.

Endpoint: `tools/call`. Accepts `{name, arguments}` and routes to `task.step(Action(name, arguments))`, returning the serialized `EnvironmentOutput` (observation, done flag, reward if terminal, info dict). A call after `done=True` raises a protocol error; the client must call `tools/list` on a fresh session to start a new episode.

Server lifecycle. `Benchmark._setup()` starts the server (one per benchmark, shared across tasks); `Benchmark.close()` tears it down. Individual tasks register and deregister their endpoints on `reset()` and episode end. The server address is published in `RuntimeContext` so remote workers can connect without further coordination.

B Per-benchmark task count, license, infra requirements

Table 2: The CUBE registry at submission.

CUBE	Modality	# Tasks	Backend
WorkArena	Web	333	Live enterprise web app
WebArena-Verified	Web	812	Shared 6-service Docker stack
MiniWoB	Web	125	Local browser
BrowseComp	Web	1,266	Browser
OSWorld	CUA	368	Ubuntu 22.04 VM
Windows Agent Arena	CUA	154	Windows 11 VM
SWE-bench Verified	SWE	500	Per-task Docker
SWE-bench Live	SWE	1,895	Per-task Docker
Terminal-Bench	Terminal	89	Per-task Docker
DRBench	Research	100	Per-task Docker (Nextcloud, Mattermost, IMAP)
Total		**5,642**	

C Judge output schema and blame taxonomy

Output schema. Each `JudgeOutput` carries five fields: an *outcome* (`success`, `success_lucky`, `almost`, `failure`, `should_have_been_rewarded`); a *primary blame* from the closed taxonomy plus optional *secondary blames*; *evidence*, the step-indexed verbatim quotes; a *hypothesis* of what would fix the failure class; and 0–5 confidence scores on both blame and hypothesis. A free-form *analysis* field placed first acts as a reasoning scratchpad before the structured fields are committed.

Blame taxonomy. Ten benchmark-agnostic categories grouped into three buckets. **Agent-side**: `model_capability` (the LLM lacked the reasoning, knowledge, or planning to solve the task); `agent_scaffolding` (the agent loop, prompt, or context management caused the failure); `submission_format` (the agent reached a solution but failed to submit it through the registered channel). **Tool-side**: `tool_failure` (a tool wrapper raised or returned an unexpected error); `action_space_limited` (no available action could express the correct solution); `insufficient_observation` (the rendered observation hid information the agent needed). **Benchmark-side**: `env_failure` (infrastructure crash outside the agent's and tool's control);

`task_unclear` (the task description is ambiguous or contradictory); `eval_brittle` (the evaluator rejected an acceptable solution). A residual `none` category is assigned on clean success or when evidence is insufficient for any attribution.

D Drift sources in dynamic-agent benchmarks

Five sources of drift recur across the corpus and motivate the reliability framing of Appendix D.

Scaffolding sensitivity. Pass rate depends heavily on the agent loop, the tool wrappers, the prompt boilerplate, and the exact context-management strategy. Two scaffolds calling the same model can differ by tens of points on the same benchmark; cross-benchmark comparisons that mix scaffolds are not measuring the model.

Prompt tailoring. Many published numbers use a prompt or chain-of-thought structure tuned for a specific benchmark, sometimes with in-context examples drawn from its training split. The reported number is then a joint property of the model and the prompt-engineering effort, not of the model in a generalist setting.

Model version drift. Frontier APIs are continuously retrained and behaviour-tuned. A model identifier (`claude-sonnet-4-6`, `gpt-5.4`) names a moving target whose behaviour at the time of a published number may not be reproducible months later.

Closed-source scaffolds. Strong leaderboard numbers are sometimes reported by labs whose scaffold, tool stack, and prompt are not released, in contexts where headline numbers carry commercial weight. These results are not reproducible by construction.

Live-component drift. Several benchmarks in the corpus depend on live external services or system state that change without notice: WorkArena targets a live enterprise web application; OSWorld and Windows Agent Arena depend on OS revisions; underlying browser-automation libraries (`playwright`, `pyautogui`) ship breaking updates between minor versions. Two runs weeks apart against the same task set can produce different ground-truth pass rates.

E Judge meta-analysis

For every episode that received three independent judge invocations we measure *primary-blame agreement* (do all three judges choose the same blame category?) and *outcome agreement* (do all three agree on pass / almost / fail?). Tables 3 and 4 report the counts and rates for the two benchmarks where triple-judge runs were completed: SWE-bench Verified (Sonnet 4.6, n=162; GPT-5.4, n=204) and SWE-bench Live (Sonnet 4.6, n=83).

Table 3: Primary-blame agreement across three independent judges. *3/3*: all three choose the same category. *2/3*: a two-judge majority. *All differ*: no two judges agree. $\geq 2/3$ subsumes both 3/3 and 2/3 rows.

Benchmark	Model	n	3/3	2/3	All differ	3/3 %	$\geq 2/3$ %
SWE-bench Verified	Sonnet 4.6	162	117	40	5	72.2	96.9
SWE-bench Verified	GPT-5.4	204	173	27	4	84.8	98.0
SWE-bench Live	Sonnet 4.6	83	64	19	0	77.1	100.0

Table 4: Outcome agreement (pass / almost / fail) across three independent judges. Column definitions identical to Table 3.

Benchmark	Model	n	3/3	2/3	All differ	3/3 %	$\geq 2/3$ %
SWE-bench Verified	Sonnet 4.6	162	114	46	2	70.4	98.8
SWE-bench Verified	GPT-5.4	204	126	77	1	61.8	99.5
SWE-bench Live	Sonnet 4.6	83	77	6	0	92.8	100.0

Takeaways. Majority ($\geq 2/3$) agreement is **97–100 %** across every condition for both blame and outcome, meaning the modal vote is almost always stable and reliable as a signal. Full unanimous agreement is somewhat lower — **62–85 %** for blame and **62–93 %** for outcome — which is expected: the taxonomy contains adjacent categories (*model_capability* vs. *agent_scaffolding*) that can legitimately both apply to the same failure, and the outcome three-way split (pass / almost / fail) places hard boundaries on a continuous quality gradient.

The hardest condition is SWE-bench Verified outcome for GPT-5.4, where unanimous agreement falls to 61.8 %. We attribute this to the high overall pass rate on that benchmark ($\approx 58\,\%$): many GPT-5.4 episodes produce partial fixes that sit ambiguously between *almost* and *fail*, a genuinely difficult judgement call even for human annotators. Notably, the *all-differ* rate (complete three-way disagreement) stays below 2.5 % in every condition, confirming that true confusion is rare and that majority voting is a robust aggregation strategy.

F Judge case studies

Table 5 shows a representative case in which all three judges unanimously converge, illustrating both the evidence-grounding mechanism and the blame attribution in practice. The episode is drawn from the SWE-bench Verified run (§6); the judge had access to the full step-indexed trajectory and the privileged evaluation output.

Table 5: Three independent judge outputs for episode `astropy__astropy-14365` (SWE-bench Verified, outcome *almost*). All three judges unanimously agree on primary blame and intervention; high-confidence evidence quotes pinpoint the unfixed downstream `if v == "NO":` comparison that the agent read but failed to connect to its own regex fix. Evidence quotes are verbatim from the judge output; long code blocks are truncated at [...] to fit the column.

Field	Judge 1	Judge 2	Judge 3
Outcome	*almost*	*almost*	*almost*
Primary blame	`model_capability` (conf. 4/5)	`model_capability` (conf. 3/5)	`model_capability` (conf. 4/5)
Secondary	`agent_scaffolding`	–	`agent_scaffolding`
Summary	Added `re.IGNORECASE` to QDP command regex but missed downstream `if v == "NO":` null-marker check; verified only syntax compilation, never ran the test suite.	Fix incomplete: `re.IGNORECASE` applied to command recognition; null-value sentinel `"NO"` still compared case-sensitively; crashes on lowercase `"no"` with `ValueError`.	Regex fix correct for command recognition; identical case-sensitivity bug in data-value processing loop not connected; `final_step` called without running any tests.

Key evidence	*step 8* (verified):
	`for v in line.split(delimiter):`
	`  if v == "NO":`
	`  ...`
	`  else: [...]`
	step 16 (verified):
	`ValueError: could not convert string to float: 'no'`
		step 56 (verified, match 0.88):
		`ValueError: could not convert string to float: 'no'`
		`astropy/io/ascii/qdp.py:317: ValueError`
			step 8 (verified):
			`for v in line.split(delimiter):`
			`  if v == "NO":`
			`  ...`
			`  else: [...]`
			step 15 (verified):
			`ACTION final_step: {}`

Hypothesis	Change `"NO"` to `v.upper() == "NO"` throughout; mandate `pytest` run on the failing test before submit.	Run the actual failing test before submitting; would reveal the secondary failure immediately.	Add mandatory post-fix validation step (run failing reproduction case) to the agent workflow.
Intervention	`prompt_change` (conf. 3/5)	`prompt_change` (conf. 3/5)	`prompt_change` (conf. 4/5)

Table 6: Three independent judge outputs for episode `conan-io__conan-18028` (SWE-bench Live, outcome *failure*). The agent located `graph_builder.py` and the `skip_test` check at step 82, then immediately fell into a deterministic loop, re-issuing the same `grep` command 53 consecutive times and hitting `MAX_STEPS_REACHED` without writing a single line of code. All three judges agree on `scaffold_change` as the intervention; two of three assign primary blame to `agent_scaffolding`. Evidence quotes are verbatim from the judge output; long action strings are truncated at [...] to fit the column.

Field	Judge 1	Judge 2	Judge 3
Outcome	*failure*	*failure*	*failure*
Primary blame	`agent_scaffolding` (conf. 5/5)	`model_capability` (conf. 4/5)	`agent_scaffolding` (conf. 4/5)
Secondary	`model_capability`	`agent_scaffolding`	`model_capability`
Summary	Spent all 150 steps in read-only exploration, running identical `grep` commands dozens of times without writing a single code edit or calling `final_step`.	Spent all 150 steps in `grep`/`cat` exploration; from step 80 fell into a tight loop re-running the same `grep` with identical output each time.	Located `tools.graph:skip_test` in `graph_builder.py` at step 81, then immediately fell into a deterministic loop: same `grep` command 53 consecutive times, 0 code edits, `MAX_STEPS_REACHED`.

Key evidence	*steps 99, 101, 103* (verified, match 0.86):
	`ACTION bash: {... grep -rn "skip_test|tools.graph:skip .../graph_builder.py}`
	steps 295, 297, 299 (verified, match 0.90):
	`ACTION bash: {... grep -rn "node|test\b" conans/conan/ -include="*.py" | grep -v "test/"}`
		step 81 (verified, match 0.86):
		`ACTION bash: {... grep -rn "skip_test|tools.graph:skip_ .../graph_builder.py}`
		step 121 (verified, match 0.86):
		`ACTION bash: {... grep -rn "skip_test|tools.graph:skip_ .../graph_builder.py}`
			step 82 (verified):
			`216: skip_test = node.conanfile.conf .get("tools.graph:skip_test",...)`
			`222: if skip_test and require.test:`
			step 83 → 151 (loop begins):
			`ACTION bash: {... grep -rn "skip_test|tools.graph:skip_test" .../graph_builder.py}`
			metadata (step 0):
			`"status": "MAX_STEPS_REACHED", "prompt_tokens": 6430181, "cached_tokens": 6267737`

Hypothesis	Adding loop detection (block on N consecutive identical tool calls) and context compaction at a reasonable threshold would prevent this class of failure.	A scaffold with loop detection would have interrupted the cycle and forced a different action; the agent had located the relevant code but lacked the push to synthesize an edit.	Full history, temperature 0, and no compaction create a deterministic feedback loop once repetition starts; aborting after K consecutive identical actions would break the cycle.
Intervention	`scaffold_change` (conf. 5/5)	`scaffold_change` (conf. 3/5)	`scaffold_change` (conf. 5/5)

Table 7: Three independent judges on three WAA episodes (ep87, ep88, ep151; all VLC-domain, all scored 0.0), working on disjoint trajectories with no shared context. All three converge on the same eval-side root cause: the vlcrc path string passed to execute_python_command is re-parsed as Python source, silently collapsing \v to U+000B (vertical tab) and corrupting the path before get_file() sees it. A one-line os.path.join fix on the VM side flipped all five affected episodes from 0.0 to 1.0 on a single resumed run.

Field	Judge / ep87	Judge / ep88	Judge / ep151
Outcome	*should_have_been_rewarded*	*should_have_been_rewarded*	*should_have_been_rewarded*
Primary blame	`eval_brittle` (conf. 3/5)	`eval_brittle` (conf. 3/5)	`tool_failure` (conf. 2/5)
Task	Set VLC recording folder to Downloads	Set VLC recording folder to Desktop	Disable "Resize interface to video size" in VLC
Summary	Agent completed navigation (Preferences → [. . .] → Downloads → Save). Evaluator's `get_vlc_config` path literal re-parsed as Python; \v → U+000B corrupted `\vlcrc` to `\u000blcrc`; `get_file` 500, false `FileNotFoundError`.	Agent navigated Preferences and set Desktop as recording folder and saved. Evaluator's `get_vlc_config` path literal re-parsed as Python; \v → U+000B corrupted `\vlcrc` to `\u000blcrc`; `get_file` 500, score 0.0.	Agent navigated Preferences and unchecked 'Resize interface to video size.' Evaluator's `get_vlc_config` path literal re-parsed as Python; \v → U+000B corrupted `\vlcrc` to `\u000blcrc`; `get_file` 500, false `FileNotFoundError`.

Key evidence	*step 9* (verified):
	`[ERROR] get_file() -- C:\Users\Docker\AppData\ Roaming\u000blc\u000blcrc`
		step 9 (verified):
		`[ERROR] get_file() -- C:\Users\Docker\AppData\ Roaming\u000blc\u000blcrc`
			step 8 (verified):
			`ACTION run_pyautogui: {"code": "pyautogui.click( 784, 689) # Save"}`
			step 9 (verified):
			`[ERROR] get_file() -- Roaming\u000blc\u000blcrc`

Hypothesis	Path literal `\vlcrc` in `execute_python_command` re-parsed as Python source; \v → U+000B. Replace with `os.path.join` to eliminate escape-sequence literals.	Classic escape-sequence trap on the Python round-trip. Build path from `os.path.join` components so no \v literal survives re-parse.	`get_vlc_config` sends a Python expression to the VM; \v in `\vlcrc` collapses to U+000B silently. Switch to `os.path.join('~', ..., 'vlcrc')`.
Intervention	`eval_fix` (conf. 4/5)	`eval_fix` (conf. 4/5)	`eval_fix` (conf. 3/5)
Post-fix outcome	**0.0 → 1.0** on resume (`is_vlc_recordings_folder` score 1.0)	**0.0 → 1.0** on resume (`is_vlc_recordings_folder` score 1.0)	**0.0 → 1.0** on resume (`is_vlc_resize_video_disabled` score 1.0)

Table 8: Three independent judge outputs for episode `716a6079_ep235` (**GPT-5.4 run**, OSWorld, outcome consensus: *failure*). Task: find `secret.docx` on the VM and copy its full path to the clipboard. File is at `/home/user/Data3/List3/secret.docx`. The three judges *disagree* on root cause: Judge 1 blames the model (agent used `Ctrl+L` in Nautilus, which copies the containing *directory* path, not the full file path, and rates the outcome *almost* — a near-miss sub-category of failure); Judges 2–3 blame infrastructure (`xsel` is not installed on the VM, so the evaluator `is_in_vm_clickboard` always reads an empty clipboard regardless of agent behavior). This split illustrates a compound failure: a model-capability weakness *and* a broken infrastructure-level evaluator co-occur, making the true verdict ambiguous. Compare with Table 9 (Sonnet run, same task): Sonnet used `xclip` correctly and received a unanimous *should_have_been_rewarded*.

Field	Judge 1	Judge 2	Judge 3
Outcome	*almost*	*failure*	*failure*
Primary blame	`model_capability` (conf. 3/5)	`eval_brittle` (conf. 4/5)	`env_failure` (conf. 4/5)
Secondary	`action_space_limited`	`model_capability`	`model_capability`
Summary	Agent found correct file via Nautilus search but copied only the containing *directory* path (`Ctrl+L`+`Ctrl+C` in Nautilus address bar) rather than the full file path including `/secret.docx`. Evaluator expected `/home/user/Data3/List3/secret.docx` but agent produced only `/home/user/Data3/List3`.	Evaluator depends on `xsel` to read VM clipboard, but `xsel` is not installed (`returncode 127`, `/bin/sh: 1: xsel: not found`). This makes scoring impossible for *any* agent. Separately, agent's `ctrl+l+ctrl+c` method would produce directory path, not file path.	VM missing `xsel` binary that `is_in_vm_clickboard` relies on; evaluation command returned `/bin/sh: 1: xsel: not found` and empty output, making reward=0 inevitable regardless of agent behavior.
Key evidence	*step 29* (verified, match 0.94): `ACTION hotkey: {"keys": "ctrl+l"}` *step 31* (verified, match 0.94): `ACTION hotkey: {"keys": "ctrl+c"}` *task config*: `"expected": "/home/user/Data3/List3/secret.docx"`	*evaluator log* (match 0.34): `{'error': '/bin/sh: 1: xsel: not found\n', 'output': '', 'returncode': 127}` *evaluator log* (match 0.52): `reward=0.000000, evaluator=is_in_vm_clickboard` *step 29* (verified, match 0.94): `ACTION hotkey: {"keys": "ctrl+l"}`	*evaluator log*: `{'error': '/bin/sh: 1: xsel: not found\n', 'returncode': 127, 'output': ''}` *evaluator*: `is_in_vm_clickboard` depends on `xsel -clipboard -output`
Hypothesis	Instruct agent to use file Properties dialog or right-click 'Copy Location' to obtain full file path; `Ctrl+L` in Nautilus yields only the directory.	Install `xsel` on VM base image, or update evaluator to fall back to `xclip -o -selection clipboard`; also fix agent clipboard strategy.	VM image (cube-ubuntu-22-04 v1.0.0) does not include `xsel`; install `xsel` or switch evaluator to `xclip -o` or `wl-paste` to fix all clipboard-check tasks.
Intervention	`prompt_change` (conf. 3/5)	`eval_fix` (conf. 5/5)	`infra_fix` (conf. 5/5)

Table 9: Three independent judge outputs for episode `716a6079_ep235` (**Sonnet run**, OS-World, outcome consensus: *should_have_been_rewarded*). *Same task as Table 8.* Unlike GPT-5.4 (which used `Ctrl+L` in Nautilus and copied only the directory path), the Sonnet agent correctly used `xclip -selection clipboard` in a terminal to copy the full file path `/home/user/Data3/List3/secret.docx`. All three judges agree: the correct agent action was irrelevant because the evaluator's clipboard-reading command (`xsel -clipboard -output`) is not installed on the VM, so the clipboard always reads as empty. This cross-model pair is direct evidence that the same broken evaluator can produce a *model-capability failure* verdict for a weaker agent and a *should_have_been_rewarded* verdict for a stronger one — masking genuine capability differences behind infrastructure noise.

Field	Judge 1	Judge 2	Judge 3
Outcome	*should_have_been_rewarded*	*should_have_been_rewarded*	*should_have_been_rewarded*
Primary blame	`eval_brittle` (conf. 4/5)	`eval_brittle` (conf. 4/5)	`eval_brittle` (conf. 4/5)
Secondary	—	`model_capability`	`env_failure`
Summary	Agent correctly found the file at `/home/user/Data3/List3/secret.docx` and used `xclip` to copy its path to the clipboard, but evaluator's clipboard-reading command (`xsel`) was not installed on the VM (`returncode 127`), causing it to always see an empty clipboard and return reward=0 regardless of agent actions.	Agent correctly identified and copied file path using `xclip -selection clipboard`, then called `done()`. Evaluator failed with `/bin/sh: 1: xsel: not found` (`returncode 127`) because `xsel` is not installed on the VM.	Agent correctly found `secret.docx` and copied its path to clipboard using `xclip`. Evaluator uses `xsel -clipboard -output` to verify; `xsel` is not installed (`returncode 127`), so comparison always sees empty string.
Key evidence	*evaluator log* (match 0.31): `{'error': '/bin/sh: 1: xsel: not found\n', 'returncode': 127, 'output': "}` *agent reasoning, step 9*: "The path `/home/user/Data3/List3/secret.docx` has been copied to the clipboard using `xclip -selection clipboard`."	*agent reasoning, step 7*: "The file has been found at `/home/user/Data3/List3/secret` *evaluator log* (match 0.31): `{'error': '/bin/sh: 1: xsel: not found\n', 'returncode': 127}`	*step 15* (match 0.92): `ACTION typing: {"text": "echo -n \"/home/user/Data3/List3/secret.docx\" | xclip -selection clipboard"}` *evaluator log*: `xsel: not found, returncode 127`
Hypothesis	VM image does not have `xsel` installed; install `xsel` or update evaluator to use `xdotool getclipboard` or `xclip -o`.	Install `xsel` on VM base image, or update evaluator to fall back to `xclip -o -selection clipboard`.	VM has `xclip` (installed) but not `xsel`; evaluator's getter always fails; fix evaluator to use `xclip -o -selection clipboard`.
Intervention	`infra_fix` (conf. 5/5)	`infra_fix` (conf. 5/5)	`eval_fix` (conf. 5/5)